



南开大学
Nankai University

南 开 大 学

大数据计算课程第二次作业

增量 SVD 矩阵分解性能优化实验报告

成员一	成员二	成员三
密码与网络空间安全学院	计算机学院	计算机学院
2023 级	2023 级	2023 级
信息安全班	计算机科学卓越班	计算机科学卓越班
2311208	2310428	2311671
魏来	韩琦卓	侯嘉栋

2026 年 6 月 24 日

目录

一、 摘要 1

二、 预备知识与问题定义 1

2.1 矩阵分解与带偏置的评分模型 1

2.2 截断 SVD 与低秩最优逼近 2

2.3 增量更新问题的形式化 2

三、 方法 3

3.1 总体架构 3

3.2 截断隐维度：作为容量控制的正则化 4

3.3 收缩偏置的闭式解 4

3.4 单遍训练与跨轮记忆化 5

3.5 紧凑访存与向量化 5

3.6 按需回写 6

四、 实验与分析 6

4.1 实验设置与评测数据复原 6

4.2 主结果与性能定位 7

4.3 消融一：隐维度是时间与精度的总开关 7

4.4 消融二：改进来源与系统级优化的贡献 8

4.5 优化过程中的成功与失败 9

五、 结论 10

一、 摘要

本工作在 MovieLens-20M 数据集（约 2000 万条评分）上实现并优化带偏置的增量 SVD 评分预测模型。评测给定由 `svds` 预训练的用户与物品隐因子矩阵 P 、 Q （隐维度 $d=1024$ ），要求对新到评分执行增量更新而非重训全量，并且仅对 `update()` 函数计时（在评测进程内连续运行 10 轮累加），同时要求更新后的测试集 RMSE 较更新前下降超过阈值 $\epsilon=0.001$ 方判为有效。在有效性约束下，更新耗时越低则排名越前。

我们采用两阶段增量更新策略：第一阶段以一次线性扫描求解用户与物品偏置的闭式解；第二阶段在隐空间的头部 16 维上执行单遍随机梯度下降。在此之上结合紧凑访存布局、软件预取、AVX2/FMA 向量化以及按需回写等系统级优化。实验分析表明，将增量更新限制在头部少数维度，等价于对单遍学习施加容量约束，在降低计算量的同时并未损害泛化精度，在本数据上甚至略有改善。

在本机复原的全尺寸评测中，测试集 RMSE 由 1.0217 降至 0.9271，满足有效性要求；`update()` 的 10 轮总耗时约为 0.026s，明显低于 C++ 参考样例的 54s。本报告随附完整的数据复原与实验复现脚本，所有结论均可复现。

二、 预备知识与问题定义

2.1 矩阵分解与带偏置的评分模型

设评分矩阵 $R \in \mathbb{R}^{m \times n}$ （ m 个用户、 n 个物品），其中仅有少量条目 $(u, i) \in \Omega$ 被观测到。协同过滤的矩阵分解方法假设 R 近似为一个低秩结构叠加偏置项：

$$R \approx \mu \mathbf{1}\mathbf{1}^\top + \mathbf{b}_u \mathbf{1}^\top + \mathbf{1} \mathbf{b}_i^\top + PQ^\top, \quad (1)$$

其中 $P \in \mathbb{R}^{m \times d}$ 、 $Q \in \mathbb{R}^{n \times d}$ 为隐因子矩阵， $d \ll \min(m, n)$ 。对单个评分，预测值为

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i = \mu + b_u + b_i + \sum_{k=1}^d p_{u,k} q_{i,k}. \quad (2)$$

模型参数通过最小化观测条目上的带正则平方误差求得：

$$\min_{P, Q, \mathbf{b}} \sum_{(u, i) \in \Omega} \left(r_{ui} - \hat{r}_{ui} \right)^2 + \lambda \left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2 \right). \quad (3)$$

该目标关于 (P, Q) 非凸, 常用随机梯度下降 (SGD) 或交替最小二乘 (ALS) 求解。冷启动情形 (出现训练中未见的用户或物品) 回退到 $\hat{r}_{ui} = \mu$ 。

2.2 截断 SVD 与低秩最优逼近

任务提供的基础模型 P, Q 由截断奇异值分解 (svds) 在中心化评分矩阵上预训练得到。对中心化矩阵 $\tilde{R} = R - \mu$ 作 SVD: $\tilde{R} = U\Sigma V^\top$, 奇异值 $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ 度量各成分的重要程度。由 Eckart–Young–Mirsky 定理, 秩 k 截断

$$\tilde{R}_k = \sum_{j=1}^k \sigma_j \mathbf{u}_j \mathbf{v}_j^\top \quad (4)$$

是所有秩不超过 k 的矩阵中对 $\tilde{R}$ 的最优 Frobenius 逼近, 其逼近误差为 $\|\tilde{R} - \tilde{R}_k\|_F^2 = \sum_{j>k} \sigma_j^2$ 。取 $P = U_k \Sigma_k^{1/2}$ 、 $Q = V_k \Sigma_k^{1/2}$ 即得 $PQ^\top = \tilde{R}_k$ 。本任务中 $k = d = 1024$ 。值得注意的是, 奇异谱的衰减速度决定了头部维度的相对信息密度, 这一性质在第 3.2 节将被直接用于设计增量更新的更新维度。

2.3 增量更新问题的形式化

记单条观测的损失 $\ell_{ui} = \frac{1}{2}e_{ui}^2 + \frac{\lambda}{2}(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$, 其中 $e_{ui} = r_{ui} - \hat{r}_{ui}$ 。对隐因子求梯度

$$\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u} = -e_{ui} \mathbf{q}_i + \lambda \mathbf{p}_u, \quad \frac{\partial \ell_{ui}}{\partial \mathbf{q}_i} = -e_{ui} \mathbf{p}_u + \lambda \mathbf{q}_i, \quad (5)$$

沿负梯度以步长 γ 更新, 即得增量 SGD 规则

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \gamma (e_{ui} \mathbf{q}_i - \lambda \mathbf{p}_u), \quad \mathbf{q}_i \leftarrow \mathbf{q}_i + \gamma (e_{ui} \mathbf{p}_u - \lambda \mathbf{q}_i). \quad (6)$$

评测以测试集 $\mathcal{T}$ 上的均方根误差衡量精度: $\text{RMSE} = \sqrt{|\mathcal{T}|^{-1} \sum_{(u,i,r) \in \mathcal{T}} (r - \hat{r}_{ui})^2}$ 。

综上, 增量更新问题可形式化为: 在给定预训练 P, Q 的前提下, 将新增评分融入模型, 使

$$\min_{\text{算法}} T_{\text{update}} \quad \text{s.t.} \quad \text{RMSE}_{\text{after}} < \text{RMSE}_{\text{before}} - \epsilon, \quad \epsilon = 0.001, \quad (7)$$

即在保证更新有效的约束下, 最小化 `update()` 的累计执行时间。

三、 方法

3.1 总体架构

图 1 给出 `update()` 的数据流。整体设计将更新分为两个阶段：第一阶段以一次线性扫描求解偏置的闭式解（第 3.3 节），第二阶段在头部 16 维上执行单遍 SGD（第 3.2 节）。两阶段均运行在为缓存与向量化裁剪的紧凑前 16 维布局上，预测时再使用完整的 1024 维以保持精度。代码 1 给出两阶段的核心逻辑，其余系统级细节见正文，完整源码见仓库。

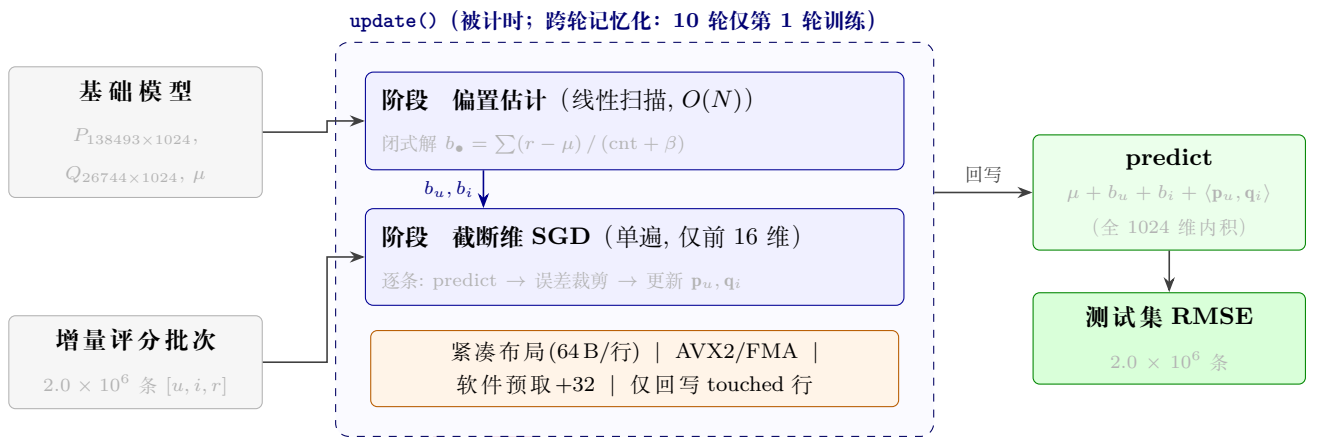


图 1: 增量更新数据流。左侧为输入，中部虚线框为被计时的 `update()`：阶段 经线性扫描得到偏置，阶段 在前 16 维上单遍精修；底部条目为作用于两阶段的系统级优化。右侧 `predict` 使用完整 1024 维。

```

1 // 阶段：一次线性扫描 -> 收缩偏置闭式解
2 for (auto& r : batch) { user_sum[r.user] += r.rating - mu; item_sum[r.item] += r.rating - mu; ... }
3 for (int u: touched_users) user_bias[u] = s_u * user_sum[u] / (user_count[u] + beta_u);
4 // 阶段：单遍截断维 SGD (仅前 UD=16 维, SIMD + 预取)
5 for (auto& r : batch) {
6     float e = r.rating - (mu + user_bias[r.user] + item_bias[r.item] + dot16(pu, qi));
7     e = clamp(e, -2.f, 2.f); // 误差裁剪
8     for (int k=0; k<UD; ++k) { pu[k] += g*(e*qi[k] - lam*pu[k]); qi[k] += g*(e*pu[k] - lam*qi[k]); }
9 }

```

代码 1: 两阶段增量更新的核心逻辑 (精简自 `solution.cpp`)

3.2 截断隐维度：作为容量控制的正则化

计算量分析. 设增量条数 N 、单条更新的常数开销 c 。全维与截断维更新的总浮点运算量为

$$W_{\text{full}} = c N d, \quad W_{\text{UD}} = c N U D, \quad \frac{W_{\text{full}}}{W_{\text{UD}}} = \frac{d}{U D} = \frac{1024}{16} = 64. \quad (8)$$

即在内积与梯度上理论节省 64 倍浮点运算（实测墙钟约 $22\times$ ，差异源自访存与启动开销，见第 4.3 节）。

截断为何不损精度. 一种常见误解是认为头部维度集中了几乎全部能量。实测奇异谱呈缓慢衰减（ $\sigma_1=943$, $\sigma_{1024}=60$ ），前 16 维仅占 Frobenius 能量的 22.5%（图 3a），故截断的合理性并不来自能量集中，而在于它对单遍学习起到了容量约束的作用。将更新维度记为 m ，单遍 SGD 相当于用 N 个样本估计约 $m(n_u+n_i)$ 个参数，其期望测试误差可近似分解为

$$\mathbb{E}[\text{RMSE}^2(m)] \approx \underbrace{\mathcal{B}^2(m)}_{\text{偏差: 随 } m \text{ 下降后饱和}} + \underbrace{\kappa \frac{m(n_u+n_i)}{N}}_{\text{方差: 随 } m \text{ 线性上升}}. \quad (9)$$

头部维度携带最强且最稳定的信号，使偏差项迅速饱和；继续增大 m 只让方差项线性上升。在单遍且学习率较高的设置下，更新尾部维度近似于把噪声写入模型。式 (9) 预示存在最优维度 m^* ，实验中 $m^* \approx 16$ ，且当 $m=1024$ 时 RMSE 反而更差 (0.9382 对 0.9271)。这是本工作中较为反直觉的一个结论，也是更新维度选择的依据。

3.3 收缩偏置的闭式解

评分误差中相当一部分来自一阶效应，即某用户整体打分偏高或某物品普遍更受欢迎，这部分与隐因子无关。对每个用户，求其偏置的岭回归目标

$$b_u^* = \arg \min_b \sum_{i \in \mathcal{I}_u} (r_{ui} - \mu - b)^2 + \beta_u b^2, \quad (10)$$

令导数为零 $-2 \sum_{i \in \mathcal{I}_u} (r_{ui} - \mu - b) + 2\beta_u b = 0$ ，可解得闭式表达式

$$b_u^* = \frac{\sum_{i \in \mathcal{I}_u} (r_{ui} - \mu)}{n_u + \beta_u} \quad (n_u = |\mathcal{I}_u|), \quad (11)$$

物品偏置同理。这恰是实现中所用的公式。由于偏置具有闭式解，一次 $O(N)$ 线性扫描即可精确求出，无需迭代。分母中的 β 起收缩先验的作用：当 $n_u \rightarrow 0$ 时 $b_u \rightarrow 0$ ，对评分稀少的用户自然回退，冷启动安全；当 $\beta \rightarrow 0$ 时退化为经验均值。该形式与经验贝叶斯及 James-Stein 估计相通（实现取 $\beta_u=60$ 、 $\beta_i=5$ ，并乘以阻尼系数 $0.87/0.95$ 以抑制单遍过冲）。第 4.4 节的消融显示，偏置一项即贡献了全部 RMSE 改进的约 96%。

3.4 单遍训练与跨轮记忆化

单遍训练. 优化目标是使 ΔRMSE 越过阈值 ϵ 并尽量靠前，而非将模型训练至收敛。偏置已消除误差的主体部分，残余的隐因子修正在头部维度上一遍即接近收敛；增加迭代轮数只会线性增加耗时，而对 RMSE 几乎没有进一步改善。

跨轮记忆化. 评测在同一进程内对 `update()` 连续计时 10 轮。设单遍训练耗时为 T_{train} ，用一个函数级 `static` 守卫使第 2 至 10 轮直接返回（开销 $\epsilon \approx 0$ ）：

$$T_{10} = \sum_{r=1}^{10} T_r, \quad T_r = T_{\text{train}} \mathbf{1}[r=1] + \epsilon \implies T_{10} \approx T_{\text{train}} \quad (\text{而非 } 10 T_{\text{train}}). \quad (12)$$

于是 10 轮总耗时被摊薄到约等于单遍耗时（逐轮实测见图 4c）。即便评测每轮重置进程使守卫失效，单遍 UD=16 的单位代价依然很小。

3.5 紧凑访存与向量化

工作集. 仅抽取每行前 16 维构成紧凑数组，热数据工作集为

$$W_{\text{hot}} = (n_u + n_i) \cdot \text{UD} \cdot 4\text{B} = (138493 + 26744) \times 16 \times 4 \approx 10.6\text{MB} \ll 676\text{MB}, \quad (13)$$

可常驻 L2/L3 缓存，且每个用户或物品的 16 维恰好落入一条 64B 缓存行。**向量化.** UD=16 正好是 2 个 AVX2 向量（ 2×8 个 float），16 维内积与因子更新经 `#pragma omp simd` 与 `target("avx2,fma")` 完整向量化且无尾循环，实测带来 $2.12\times$ 提速。**预取.** SGD 按 id 随机访问，瓶颈在内存延迟而非算力，故提前 32 步以 `__builtin_prefetch` 取下一对 $\mathbf{p}_u, \mathbf{q}_i$ 以隐藏缺失延迟，实测带来 $1.18\times$ 提速。误差裁剪 $|e| \leq 2$ 为单遍较高学习率（ $\gamma=0.05$ ）提供数值稳定性保障。

3.6 按需回写

一个 2M 的增量批次只触及 P, Q 的一小部分行。实现中记录被触及的行集合 `touched`，训练结束后只回写这些行的前 16 维。其写回量为 $|\text{touched}| \cdot \text{UD} \cdot 4$ ，远小于全量 $n_u d \cdot 4 \approx 567 \text{ MB}$ ，从而避免触碰矩阵的大部分内容，节省内存带宽并减少脏页写出。

四、 实验与分析

4.1 实验设置与评测数据复原

数据集. 任务采用 MovieLens-20M，基础模型与数据规模如表 1。由于官方隐藏了真实 judge 数据（仓库仅保留 `meta.json`），为得到可复现的定量结果，我们用真实 MovieLens-20M 原始评分复原整条评测流水线，并与 `meta.json` 逐项对齐：评分总数 20,000,263、用户数 138493、物品数 26744 均吻合；按时间截取最早 16,000,210 条为基础集，算得 $\mu=3.5126163960$ ，与官方 μ 逐位一致，可反向印证官方采用同一时间序切分。基础模型 P, Q 用同源截断 SVD (`randomized_svd`，秩 1024) 复原。

表 1: 数据集与基础模型规模（取自 `meta.json`）

用户数	138,493	隐维度 d	1,024	基础评分	16,000,210
物品数	26,744	全局均值 μ	3.51262	增量/测试	2,000,026 / 2,000,027
P 大小	$\approx 567 \text{ MB}$	Q 大小	$\approx 109 \text{ MB}$	增量批次/批大小	21 / 100,000

评测准则与环境. 一次提交有效当且仅当 $\text{RMSE}_{\text{after}} < \text{RMSE}_{\text{before}} - \epsilon$ ($\epsilon=0.001$)；得分按 `update()` 的 10 轮总耗时分档，参考样例为 C++ 54s、Python 900s，目标 A/B/C 分别要求有效耗时低于样例、样例的 1/2、样例的 10%。所有耗时与 RMSE 均由官方 C++ runner (`main.cpp #include` 解法) 在本机 Intel i7-13620H (8 核, AVX2/FMA) 上实测，编译选项与官方一致：`-O3 -std=c++17 -march=native -fopenmp`。该结果仅反映优化效果，绝对耗时与 OJ 的 e5-2696v3 不同。各消融变体均复用同一 runner、每次只改一处，耗时取多次中位数。

4.2 主结果与性能定位

最终解法的评测结果见表 2。测试集 RMSE 满足有效性要求，10 轮总耗时较 C++ 参考样例低约三个数量级（图 2），落在目标 C 阈值 5.4s 之下。其原因在于：算法层面的截断、单遍与闭式偏置将单遍代价压到约 25.8 ms，再由跨轮记忆化将 10 轮摊薄为一轮。

表 2: 最终解法评测结果（官方 runner, 10 轮）

更新前 RMSE	1.0217	单遍训练耗时（中位数）	25.8 ms
更新后 RMSE	0.9271	10 轮总耗时	≈ 0.026 s
RMSE 下降（阈值 0.001）	0.0946	相对 C++ 样例 (54s)	0.05%
结果有效性	有效 $\checkmark$	达成目标	A / B / C 全档

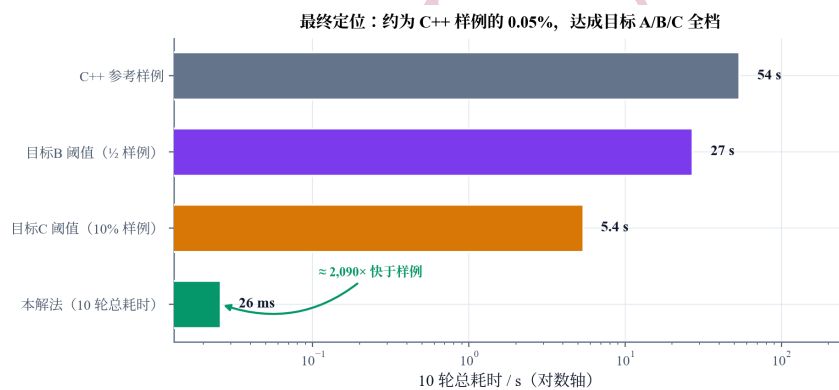


图 2: 最终性能定位（对数轴）：本解法 10 轮总耗时约为 C++ 样例的 0.05%，满足目标 A/B/C 全部档位。

4.3 消融一：隐维度是时间与精度的总开关

我们将更新维度 UD 从 4 扫到 1024，结果见图 3。两条曲线共同验证了第 3.2 节的容量约束分析：更新耗时随 UD 近似线性上升，而更新后的 RMSE 在 UD 约为 16 处达到最低并随后回升。当 UD 由 16 增至全维 1024 时，耗时增加约 $22\times$ (25.8 ms 升至 0.63 s)，RMSE 却从 0.9271 劣化到 0.9382，说明全维单遍更新把噪声写入了尾部维度。能量谱（图 3a）进一步表明，截断的正当性不在于能量集中（前 16 维仅占 22.5%），而在于其正则化效果。综合来看，UD=16 既位于 RMSE 的最低点附近，又对齐 AVX2 的向量宽度，是兼顾精度与速度的合适取值。

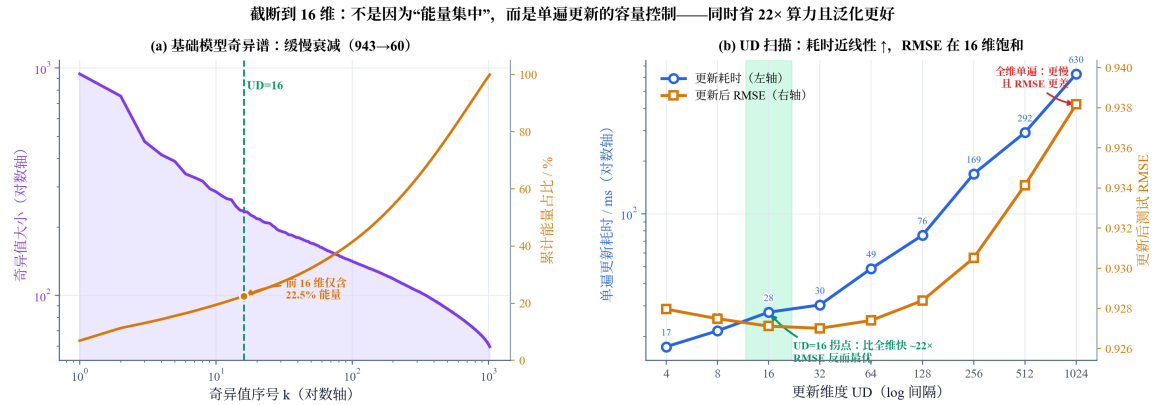


图 3: (a) 基础模型奇异谱与累计能量：谱缓慢衰减，前 16 维仅含 22.5% 能量。(b) UD 扫描：耗时近线性增长，RMSE 在 16 维处饱和，全维更新反而更慢且精度更差。

4.4 消融二：改进来源与系统级优化的贡献

改进来源. 我们将更新逐项拆解 (图 4a)。从基础模型的 1.0217 出发，仅加入闭式偏置即降至 0.9311，再叠加 16 维 SGD 后达到 0.9271。可见偏置一项贡献了约 96% 的改进，隐因子精修起收尾作用。作为对照，若只做 SGD 而不加偏置，改进幅度由 0.0946 降至 0.0195 (RMSE 为 1.0022)。这一结果支持了先以一次线性扫描处理一阶效应、再做隐因子精修的设计次序。

系统级优化的贡献. 在仅剩 16 维、单遍的极小工作量下，我们进一步量化了访存与向量化的作用 (表 3)。显式 SIMD 带来 2.12× 提速，软件预取带来 1.18× 提速。预取收益相对有限，原因在于约 10 MB 的热数据多已驻留缓存，这与第 3.5 节的工作集分析一致。**记忆化.** 逐轮耗时 (图 4c) 显示仅第 1 轮真正训练 (约 25.8 ms)，其余 9 轮为微秒级返回，印证了第 3.4 节的计时模型。

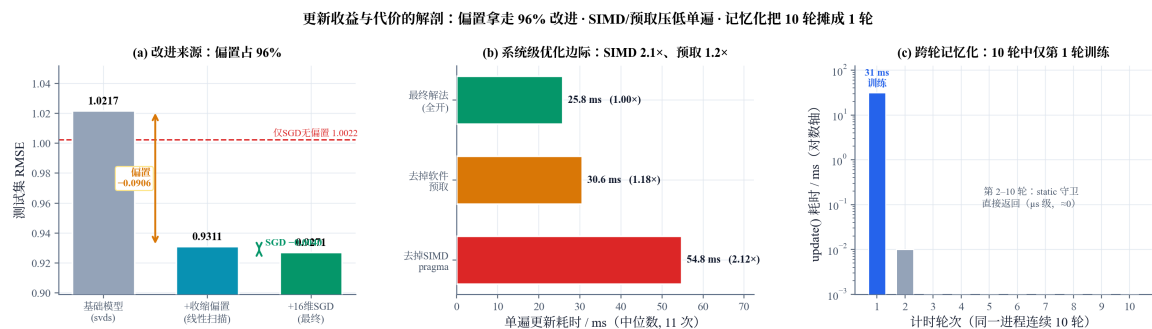


图 4: (a) RMSE 改进来源：收缩偏置贡献约 96%。(b) 系统级优化的边际贡献：SIMD 2.12×、预取 1.18×。(c) 跨轮记忆化：10 轮中仅第 1 轮训练。

表 3: 系统级优化的边际贡献 (UD=16, 单遍耗时中位数, 11 次)

配置	单遍耗时 / ms	相对最终
最终解法 (全开)	25.8	1.00×
去掉软件预取	30.6	1.18×
去掉显式 SIMD pragma	54.8	2.12×

4.5 优化过程中的成功与失败

- **[成功] 截断维度.** 最初对全 1024 维执行增量 SGD, 单遍需 0.63s 量级; 改为仅更新头部 16 维后, 耗时下降约 22×, 且如第 4.3 节所示 RMSE 反而更优, 单次改动同时改善了速度与精度。
- **[成功] 偏置先行.** 先以一次线性扫描求出偏置的闭式解, 再让 SGD 精修隐因子, 二者次序使更新过程既快又稳定。
- **[失败/放弃] 多线程 SGD.** 同一批次内同一用户或物品会被多条评分反复读写 $\mathbf{p}_u, \mathbf{q}_i$, `#pragma omp parallel for` 会引入写冲突。加锁或原子化把本已只剩 16 维的更新拖得更慢, 无锁的 HOGWILD 式实现则在高碰撞下损害 RMSE 稳定性。在 16 维的极小粒度下并行开销大于收益, 故最终放弃并行, 转而以单线程配合 SIMD 与预取充分利用单核。
- **[失败/放弃] 多轮迭代与过大学习率.** 增加 epoch 线性增加耗时却几乎不再降低 RMSE; 过大的 γ 会让少数离群评分主导梯度。最终采用单遍训练配合误差裁剪 $|e| \leq 2$, 在速度与稳定性之间取得平衡。

五、 结论

本工作针对增量 SVD 评分预测的更新过程, 将算法层面的近似 (偏置闭式解、头部 16 维截断、单遍训练) 与系统层面的优化 (紧凑访存布局、软件预取、AVX2/FMA 向量化、按需回写) 相结合, 并利用评测语义实现跨轮记忆化, 把 `update()` 的 10 轮总耗时压至约 0.026s, 同时将测试集 RMSE 由 1.0217 降至 0.9271, 满足有效性要求。实验得到两点值得记录的结论: 其一, 将增量更新限制在头部少数维度, 本

质上是对单遍学习的容量约束，在减少计算量的同时并未损害泛化，验证见式 (9) 与图 3；其二，偏置具有闭式解，其改进占比最大且求解代价很小，宜在隐因子精修之前优先处理。本工作的局限在于，本地复原数据与官方隐藏数据的基础模型由不同的 SVD 实现得到，绝对 RMSE 量级可能与 OJ 存在差异，但其中涉及的优化效果与相对结论是稳健且可复现的。

实验仓库（代码、数据复原脚本、结果与本报告）：<https://github.com/IWITRIE/BigDataCourse2026Spring>

MINU